AIA ORBIT

Your knowledge copilot · Explore · Understand · Build together

PRE-READ FOR THE EXECUTIVE COMMITTEE · AIA ORBIT

The business case in two slides

5-minute read

One idea sits behind both slides. The rules that run our life insurance business, how a surrender value is worked out, when a claim is paid or rejected, what a partial withdrawal costs, live inside decades-old AS400 programs written in COBOL. Only a few specialists can read them. So when a business owner needs to know **how something really works**, they raise a request and wait for an expert.

AIA Orbit is a next-generation knowledge copilot for our core systems. It reads the actual programs and lets anyone ask in plain English, then answers in business language, shows where the answer comes from, and goes further: it maps the impact of a change, drafts the requirement and the test scenarios, and its **coding agent** proposes the code change itself and proves it compiles. A complicated question that takes the COBOL team **7 to 15 days** to answer by checking code and logic by hand can be worked through in **minutes**, by walking a real scenario through the system. Business teams get answers on their own; specialists get their time back.

The two slides at a glance

	Slide 1 · Business and product owners	Slide 2 · AIA Singapore
Message	Ask the system itself and get the answer in business language, without waiting in a queue.	Core-system knowledge that belongs to AIA, not to a handful of people, and changes made faster and with less risk.
Core gains	Complex answers in minutes, not weeks · plain-English rules · clear impact before any change · requirements, tests and a proposed code change drafted · answers you can trace	Knowledge protected · weeks of waiting removed · faster delivery · lower change risk · stronger audit trail · specialists freed · ready for modernisation

What makes it next generation

Orbit reads the real programs, so answers are as current as the system:

Innovation	What it does for the business
Ask in plain English	Type a business question; get an answer that explains what happens, why, and who is affected, in a business or a technical view
Rules and scenarios made visible	Business rules, decision tables, flow diagrams, and step-by-step scenario walkthroughs, built from the code: the quickest route to a complicated answer
Impact before change	See what a change would touch: directly affected, also touches, downstream, with an estimated scope
From question to deliverable	Draft a functional requirement and test scenarios as ready-to-share reports
A coding agent that shows its work	Finds the exact part of a program to change, drafts the change, checks it with a real compiler, and shows current and proposed code side by side, with a plain-English summary
Evidence and safeguards	Every answer cites its source; it declines when the code does not support an answer, and blocks personal data and manipulation attempts

What we ask the Committee to take away

1. **Business owners win first.** Direct access to how our systems work changes their day; company value follows.
2. **The value is knowledge, speed and safety,** more than time saved per question.
3. **The numbers must be AIA's own.** The illustrations show how value is built; measuring real requests turns them into a business case.

SLIDE 1

What business and product owners gain

“Ask the system itself, and get the answer in business language, without waiting in a queue.”

A complicated question: 7 to 15 days by hand, minutes with Orbit

The COBOL team’s estimate for answering a complex question by checking code and logic manually is 7 to 15 days. A scenario put to Orbit is traced through the real programs step by step, so the answer comes in minutes.

A product owner’s week, before and after

	Today	With AIA Orbit
How does it work?	Raise a request; wait for the specialist to trace the code by hand (7 to 15 days if it is complicated); get a technical reply	Ask in plain English, or put a scenario to it, and get a business explanation in minutes, with the source cited
What is the rule exactly?	Rely on old documents, memory or a conversation; rules differ by who you ask	The rule is read from the live programs and shown as a plain statement, a decision table or a flow
What would a change touch?	A meeting, then days of investigation before anyone can estimate	An impact map in seconds: what is affected directly, what it also touches, what follows downstream
Writing the requirement	Start from a blank page and hope nothing was missed	A first-draft functional requirement grounded in how the system really behaves
Preparing testing	Test cases built by hand from partial knowledge	Test scenarios drafted from the real rules, including the edge cases
Making the change	A developer finds the right place in the program and writes the change by hand	A coding agent proposes the change, checks it compiles, and shows it beside the current code

Seven gains, in the product owner’s own terms

1. **Independence.** Answers to “how does this work?” no longer depend on a specialist’s diary. Ask at any time, in your own words, as often as you like.
2. **Business language, not code.** Explanations lead with what happens and why it matters to the policyholder, the claims team or finance, and name who is affected.
3. **Confidence before committing.** See the likely reach of a change before it is promised to the business, so timelines and scope are set on evidence.
4. **Better requirements and a head start on the change.** Drafts start from what the system already does, and the coding agent proposes the change for a developer to review, so fewer surprises surface in build and testing.
5. **Answers you can trust and check.** Each answer shows the programs behind it. If the code does not support an answer, Orbit says so rather than guess.
6. **Complicated problems solved by scenario.** Put a real case to Orbit, such as a surrender in year three on a policy with a loan, and watch it traced step by step. What took days of manual checking becomes a conversation.
7. **A shared language with IT.** Business and technology look at the same rules, flows and impact map, so conversations start from the same page.

Typical questions a product owner can now answer alone

- What happens to a policy when a customer surrenders, and how is the value worked out?
- Under what conditions is a claim rejected, and what does the customer see?
- If we add a new field or a new check to partial withdrawals, what else is affected?

“A question that once waited in a specialist’s queue is answered in moments, in business language, with the source shown.”

OBSERVED IN THE WORKING PROTOTYPE

SLIDE 2

What AIA Singapore gains

“Core-system knowledge that belongs to AIA, not to a handful of people, and change delivered faster and with less risk.”

Six gains for the company

Gain	What changes for AIA
1 · Knowledge protected	What the core systems do is no longer held by a few people. Experience is available to everyone, and survives retirement and role changes
2 · Faster delivery of change	Weeks of manual tracing disappear. Understanding, scoping, requirements, a proposed code change and test preparation all start sooner and from firmer ground, so product changes reach customers earlier
3 · Lower change risk	Impact is seen before work begins, reducing surprises, production incidents and rework in systems that run policies and payments
4 · Stronger audit and compliance	Rules can be explained in plain language and traced to the exact programs, supporting audit, regulator questions and product governance
5 · Specialists freed for higher-value work	Scarce experts spend less time answering repeat questions and more on engineering, modernisation and risk
6 · Ready for modernisation	Knowing exactly what the current systems do is the first step of any migration. Orbit builds that map as a by-product of daily use

The financial logic, kept simple

Value = questions answered without a specialist × time saved × value of an hour, plus faster delivery and avoided incidents

Time saved is the easiest part to measure. The larger value is usually in change delivered sooner and incidents that never happen.

Illustration	Example value
Routine questions per month needing a specialist · hours each	100 · 2 hours
Specialist and business time returned per month	**200 hours**
Complicated question: waiting time by hand (COBOL team estimate)	7 to 15 days
Complicated question: with Orbit	Minutes (to be confirmed in the pilot)

Read this as an illustration, not a forecast. The 100 questions and 2 hours are planning assumptions, and 7 to 15 days is the COBOL team’s estimate of elapsed time, not measured effort. The first step is to log how many requests of this kind really occur and how long they take, so every figure shown to the Committee rests on AIA’s own numbers.

- **Value beyond time:** one avoided production incident, or one change delivered a month earlier, may outweigh a year of time saved.

Strategic position

- **Next generation, not another document:** a living view of the systems, always as current as the code itself.
- **Beyond answers to action:** the coding agent carries a change from question to a compiler-checked proposal, always for a person to review.
- **A platform that grows:** more products and systems can be added, and the same approach serves business, audit, testing and onboarding.
- **Safe by design:** answers come only from our own code, personal data and manipulation attempts are blocked, and it explains and drafts rather than changing live systems.

FAQ

Question	Short answer
What does the coding agent do?	It finds the part of a program that must change, drafts the new code, runs a real compiler over it, fixes errors that the change introduced, and shows current and proposed code side by side with a plain-English summary. A developer reviews and decides.
Does it replace our specialists?	No. It takes the repeat questions off them, so they spend their time on the work only they can do. They remain the authority on anything Orbit cannot answer.
Can we trust the answers?	Answers come only from our own programs, with the sources shown. When the code does not support an answer, Orbit says so. Users can rate answers, and the ratings are tracked.
Will it change our live systems?	No. It explains, maps impact and drafts documents and proposed code changes for people to review. The coding agent never deploys anything.
What about customer data and misuse?	Questions containing personal data such as ID or card numbers, and attempts to manipulate the assistant, are blocked before any answer is produced, and each event is logged.
How is this different from documentation?	Documents age and are often missing. Orbit reads the real programs, so what it says matches what the system actually does.
Is the saving real, or assumed?	Assumed in the illustration above. It is to be measured against real requests before any commitment is made.
Is 7 to 15 days really the norm?	It is the COBOL team's own estimate for complicated questions answered by manual code and logic checks. It is elapsed time, not hands-on effort, and the pilot will measure it properly.
What is the downside of waiting?	Knowledge stays with a few people, change stays slow and risky, and every retirement or resignation takes understanding with it. The value above is deferred, not lost.

Figures marked as illustrations, estimates or assumptions are planning inputs to be confirmed with Finance and a baseline measurement. Product behaviour described here was observed in the current working prototype.